SOEN 321 Project Report

Members
Alex Khoury
Christopher Murdoch Egener
Hudson Lu
Jeremy Anderson
Max Pearce Basman
Tianmu Yang

Class: SOEN 321 Section U
Date: 2026-04-17

Contents

Abstract 3
Introduction 3
System Design 4
Scenario and Security Goals 4
Cryptographic Architecture 4
Authenticated Session Establishment 5
Replay Protection 5
Protected Message Exchange 5
Example Exchange 6
Implementation Code 7
AES-128 Implementation 7
RSA Signature Implementation 8
Diffie-Hellman and Session-Key Derivation 8
Communication Layer 8
Security Analysis and Discussion 9
AES-128 Security 9
RSA Security 10
RSA Attacks 10
Diffie-Hellman Security 11
Diffie-Hellman Attacks 12
Overall Hardening Measures 12
Conclusion 12
Contributions 13
References 14

Abstract

This report presents the design, implementation, and analysis of a secure messaging prototype developed for the *SOEN-321* course project. The system is intended for communication over an untrsuted netowrk and combines *Diffie-Hellman (DH)* key exchange, *RSA* digital signatures, *SHA-256* based session-key derivation, and *AES-128* encryption. The final prototype supports both a command-line demonstration and message exchange over *WebSockets*.

The report explains how these cryptographic components are combined to provide confidentiality, authentication, integrity, and session freshness. It also evaluates important limitations of the prototype, including trust in public-key distribution, replay handling, demonstration-sized *RSA* parameters, and the simplicity of the current *AES* message layer. Overall, the project shows how a layered cryptographic design can secure a communication workflow while also highlighting the gap between an educational prototype and a production-ready secure messaging system.

Introduction

Secure communication over public or otherwise untrusted networks requires more than simply hiding message contents. A practical system must also confirm the identity of the communicating parties, detect unauthorized modification of transmitted data, and resist common attacks on transmitted traffic. These requirements are especially important in messaging applications, where an attacker may monitor, intercept, or alter packets while they are in transit.

For this project, our selected scenario is secure messaging between two users communicating over an untrusted channel. The design assumes that the communication medium itself cannot be trusted to provide confidentiality or integrity. Instead, security is enforced at the endpoints. The participants authenticate one another, establish a fresh session key, encrypt messages, and verify message authenticity before decryption. This reflects a key principle in applied cryptography: security should come from protocol design and key management rather than from assumptions about the safety of the network itself [1].

To achieve these goals, the project combines several cryptographic techniques with distinct roles. Diffie-Hellman is used to establish a fresh shared secret for each session [2]. RSA signatures authenticate the exchanged handshake values and protect the signed packet contents [3]. SHA-256 derives session key material from the shared secret and session nonces [4], and AES-128 encrypts the message payload [5]. This hybrid structure is appropriate because asymmetric cryptography is well suited to authentication and key establishment, while symmetric encryption is more efficient for protecting message contents. The system

was implemented in Python as an educational prototype so that each step of the protocol can be inspected, tested, and analyzed directly.

System Design

Scenario and Security Goals

The system models two users who want to exchange confidential messages over an untrusted network. An attacker may be able to observe traffic, modify packets in transit, or attempt to impersonate one of the participants during session establishment.

Based on this threat model, the design targets four main security goals:

- **Confidentiality**, so that intercepted ciphertext does not reveal plaintext.
- **Authentication**, so that each party can verify the sender's identity.
- **Integrity**, so that unauthorized changes to a packet are detected.
- **Freshness**, so that old handshake material or packets cannot be reused without detection

Threat	Mitigation
Eavesdropping	AES-128-CBC encryption
MITM on handshake	RSA-signed DH values
Message tampering	RSA signature on ciphertext
Replay attack	Sequence numbers + session nonces
Key compromise	Forward secrecy via ephemeral DH

Cryptographic Architecture

Each participant owns an RSA key pair used for digital signatures. During session establishment, both participants also generate temporary Diffie-Hellman private values and exchange the corresponding public values. The implementation uses *RFC 3526 Group 14* 2048-bit MODP (modular exponential) parameters with generator 2 for Diffie-Hellman [2]. After the handshake completes, both parties derive the same 32-byte session-key material by hashing the shared secret together with two fresh nonces using SHA-256. The first 16 bytes of this derived value are then used as the AES-128 key for message encryption.

Component	Role in the system
Diffie-Hellman	Establishes a fresh shared secret for each session
RSA signatures	Authenticates the handshake and signs transmitted packets
SHA-256	Derives session-key material from the shared secret and nonces
AES-128-CBC	Encrypts the plaintext payload before transmission using a random IV

Component	Role in the system
Session nonces and sequence counters	Bind packets to the current session and help detect replayed messages

This design follows the structure of a hybrid secure communication system: Diffie-Hellman provides key agreement; RSA provides authentication and integrity; SHA-256 derives keying material; and AES provides efficient confidentiality for the actual message data [2], [3], [4], [5].

Authenticated Session Establishment

The session begins when the initiator generates a private Diffie-Hellman exponent a, computes the public value $A = g^a \bmod p$, creates a fresh nonce and signs the transcript containing the participant identities, the public value, and the nonce using its RSA private key. The responder verifies this signature using the initiator's RSA public key. If verification fails, the session is rejected immediately.

After successful verification, the responder generates its own private exponent b, computes $B = g^b \bmod p$, creates a second fresh nonce, and derives the shared secret $K = A^b \bmod p$. The responder then signs a second transcript containing its identity, the new public value and both nonces before sending it back. The initiator verifies this second signature and computes the same shared secret as $K = B^a \bmod p$. Session key derivation uses `SHA-256(K || nonce_initiator || nonce_responder)`, where K is the DH shared secret. This binds the key to both participants' contributions and prevents key reuse across sessions.

This signature step is essential because plain Diffie-Hellman alone is vulnerable to a man-in-the-middle attack [2]. The exchange of public values and nonce will be signed by the protocol in order to bind the handshake process with the identity of the sender and thus prevent the attacker from replacing the value and creating his own RSA signature.

Replay Protection

The packet contains a sequence number that is always growing. In each session, the maximum sequence number is stored and used by the receiver in order to discard all the packets that have a sequence number smaller than or equal to this number.

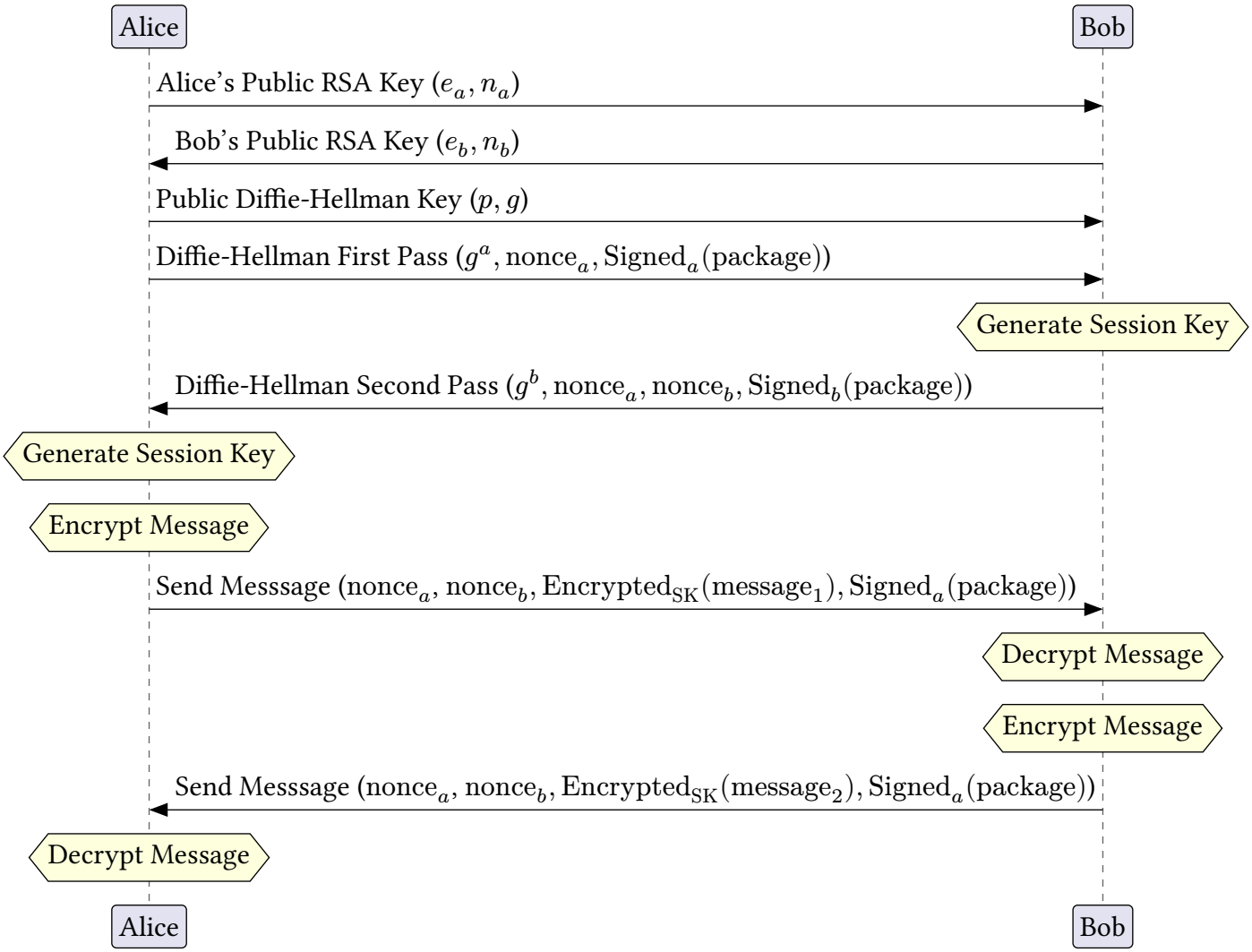
Protected Message Exchange

Once the session key has been established, the sender encrypts the plaintext message with the project's AES-128 implementation. The resulting ciphertext includes a fresh random IV so that repeated messages under the same session key do not produce identical encrypted outputs. Before transmitting, the system creates a packet header containing metadata such as the sender identity, nonces and sequence number. The sender signs the serialized header together with the ciphertext using its RSA private key.

On receipt, the receiver first verifies the signature, then checks that the packet belongs to the current session by validating the session nonces and the sequence number, and only then decrypts the ciphertext. This ordering ensures that any altered or replayed packet will be rejected before plaintext recovery is attempted.

The final implementation supports this workflow through a command-line demo and a WebSocket-based message exchange. The command-line interface supports key generation, key exchange, message encryption, message decryption, and an end-to-end demo command. The WebSocket layer exchanges the same authenticated handshake messages and signed encrypted packets over a network connection between two machines. The app layer also provides a simple real-time chat entry point for sending and receiving messages.

Example Exchange



Implementation Code

AES-128 Implementation

AES-128 is implemented as part of this project to demonstrate a symmetric encryption algorithm operating on fixed-size data blocks. AES (Advanced Encryption Standard) is a symmetric block cipher, meaning the same secret key is used for both encryption and decryption. It operates on 16-byte blocks (128 bits), and the key size is also 16 bytes. In the integrated protocol, both participants derive 32 bytes of session material from the Diffie-Hellman shared secret and two nonces using SHA-256, and the first 16 bytes are used as the AES-128 key.

The AES module includes key expansion, block encryption, block decryption, CBC-mode encryption and decryption, and PKCS7-style padding. In the current implementation, message encryption uses AES-128-CBC with a fresh random initialization vector (IV) for each message. An IV is produced by a random number generator (os.urandom) that is cryptographically strong and is concatenated to the cipher text. Every message will have its own IV such that even when messages are identical, they will end up having distinct cipher texts. This approach provides better protection for the message layer compared to an approach that simply involves encrypting individual blocks since identical messages will not result in identical cipher text for the same session key [6]. Encryption involves ten rounds of transformations of the data block. Each round strengthens security through a combination of substitution and mixing operations. The four transformations used in AES-128 are [5]:

- **SubBytes / InvSubBytes**: Each byte is replaced using a predefined S-box lookup table, which introduces non-linearity.
- **ShiftRows / InvShiftRows**: Rows of the state matrix are cyclically shifted to spread data across columns.
- **MixColumns / InvMixColumns**: Columns are mixed using arithmetic in $GF(2^8)$, providing diffusion through multiplication by a fixed mix matrix.
- **AddRoundKey**: The state is XORed with a round key derived from the original key.

The initial round (round 0) applies only **AddRoundKey**, while the final round excludes the **MixColumns** transformation [5]. Decryption applies the inverse operations in reverse order. AES-128 uses a 16-byte key which is expanded into multiple round keys through the key expansion algorithm. AES key expansion generates 44 words (from w_0 to w_{43}), which are grouped into 11 round keys: 1 initial key and 10 round keys for the remaining rounds [5]. Each word consists of 4 bytes, and each round key is composed of 4 words. The expansion process uses XOR operations and a transformation known as the g function, which consists of three steps: rotating a 4-byte word to the left by one byte, substituting each byte using the S-box, and finally XORing the first byte with the round constant [5].

RSA Signature Implementation

RSA is implemented manually to support key generation, signing, and signature verification. The key-generation routine creates two large primes, computes $n = p * q$, computes Euler's totient $\varphi(n) = (p-1)(q-1)$, and derives the private exponent d as the modular inverse of the public exponent e modulo $\varphi(n)$. The implementation uses $e = 65537$, which is a common choice because it balances efficiency and security [3].

For signatures, message bytes are first hashed with SHA-256 and converted to an integer. The signer raises that integer to the private exponent modulo n, and the receiver verifies the signature by applying the public exponent and comparing the recovered value with the expected hash. In the final system, RSA is used to authenticate both the Diffie-Hellman handshake messages and the encrypted message packets rather than to encrypt the full message payload.

Diffie-Hellman and Session-Key Derivation

The Diffie-Hellman implementation uses a standard 2048-bit MODP group with generator 2 [2]. Each participant generates a random private exponent, computes a public value, and derives the shared secret from the peer's public value. This shared secret is never sent over the network. Instead, both sides combine it with two fresh nonces and apply SHA-256 to derive the same session-key material locally.

The nonces serve two purposes. First, they help ensure that even if two sessions accidentally produce the same Diffie-Hellman shared secret, the final derived session key would still differ. Second, they contribute to freshness by binding the current session to values that should not repeat across separate connections. The session state also stores incoming and outgoing sequence counters, which are used during message exchange to detect replayed packets within the same session.

Communication Layer

The project includes multiple ways to demonstrate the protocol. The command-line interface in `main.py` supports key generation, key exchange, message encryption, message decryption, and an end-to-end demo command. The `websocket.py` module extends the same cryptographic workflow to message exchange over a network connection between two machines.

In addition, `app.py` provides a simple real-time chat entry point that starts a background listener thread and allows users to type and send messages from the terminal. The project also includes `attack_demo.py`, which demonstrates three security scenarios: a man-in-the-middle attempt on the Diffie-Hellman handshake, ciphertext tampering, and an intra-session replay attack.

During development, a separate FastAPI prototype was also created. It explored how secure message packages could be stored and retrieved through HTTP endpoints, but it remained a branch-level prototype rather than part of the final merged implementation.

Security Analysis and Discussion

AES-128 Security

AES-128 provides strong security through a combination of confusion and diffusion. Confusion is achieved through non-linear substitution using the S-box. Diffusion is achieved through ShiftRows and MixColumns, ensuring that small changes in input affect many output bits. These properties make it difficult for attackers to detect patterns or reverse the encryption process. AES-128 is widely considered secure against known practical attacks [5], [7]:

- Brute-force attacks: The key space is extremely large (2^{128}), making exhaustive search infeasible.
- Statistical attacks: Properly encrypted ciphertext should appear random and reveal little useful structure.
- Differential and linear cryptanalysis: AES is resistant in practical scenarios when implemented correctly.

Additionally, AES is efficient because of its compact and structured design. It performs well on both hardware and software systems, and it requires relatively low memory resources [5]. Despite its strong theoretical security, improper implementation can introduce vulnerabilities:

- Reuse of encryption keys
- Weak or insecure key storage
- Timing attacks (execution-time leakage)
- Cache-based side-channel attacks

In this prototype, the padded blocks are not individually encrypted. Instead, the encryption process is performed on the messages using AES-128-CBC encryption along with PKCS7 padding scheme where a new IV is used for each encrypted message [6]. The reason why repeating plain text blocks would not result in repeating cipher text blocks is that the main source of leaking patterns in ECB mode is avoided. The external RSA signature still protects packet integrity and sender authentication. A production-ready design could further strengthen this message layer by using a standard authenticated-encryption mode such as AES-GCM [8].

RSA Security

RSA provides authentication and integrity in this design by signing handshake values and encrypted packets. Its security depends primarily on the difficulty of factoring the modulus $n = p * q$ and on the correct handling of keys and padding. When RSA parameters are large enough and generated correctly, direct recovery of the private key is computationally infeasible [3].

RSA is an asymmetric public-key cryptography algorithm. It uses a trapdoor function: it is easy to compute in one direction but hard to reverse unless you know the secret key [3]. The public key is composed of n and e. In the mathematical construction, the private-key side depends on p, q, $\varphi(n)$, and d, although in the implementation the stored private key material is d together with n. The relationship between these parameters is available in the implementation section above.

RSA Attacks

In terms of security, there are two important types of attacks apart from the secret-key parameters being leaked:

- Factoring attacks, where an attacker tries to factor $n = p * q$. By recovering p and q, they can compute $\varphi(n)$ and get the private exponent d. Factoring can be attempted through brute force with trial division or through more advanced algorithms.
  - A brute-force attack can be difficult, as the complexity grows very quickly, and it is not feasible with real RSA sizes such as 2048-bit RSA.
  - More advanced factoring methods are also still computationally infeasible at proper RSA sizes.
- Protocol or key-generation failure attacks, where the weakness comes from bad parameter choices or unsafe reuse rather than from directly factoring n.
  - Twin-prime attack: where p and q are primes that are extremely close, making factorization easier than intended.
  - Common-prime attack: where the same p is reused across many public keys. In that case, one may recover $p = \gcd(n_1, n_2)$.
  - Common-modulus and related misuse attacks: when RSA parameters are reused incorrectly across users or across operations.

Other weaknesses of textbook RSA include determinism and the lack of randomized padding. Without proper padding, RSA operations are vulnerable to several practical attacks and should not be used directly [3]. In this project, RSA is used for signatures rather than for bulk message encryption, so message-recovery attacks against raw RSA encryption are less directly applicable to the final design.

However, RSA is usually used for signatures or for protecting key-establishment material rather than for full-message encryption, because it is computationally more expensive than AES [3]. We can choose a low-Hamming-weight public exponent e to speed up computations, and we can use CRT to speed up private-key operations as well.

The current prototype uses 512-bit RSA keys in its default demo mode so that the program remains fast and easy to test. This is acceptable for a classroom demonstration, but it is not secure for real deployment. In practice, RSA keys should be at least 2048 bits, and signature padding schemes such as RSA-PSS should be preferred [3].

Diffie-Hellman Security

The Diffie-Hellman portion of the protocol provides the foundation for session confidentiality by allowing both parties to derive the same shared secret without transmitting that secret directly. In plain Diffie-Hellman, the main weakness is the man-in-the-middle attack, where an active attacker replaces the public values exchanged by the two legitimate parties and establishes separate shared secrets with each victim.

This project addresses that weakness by signing the exchanged public values and both session nonces with RSA. Because the initiator and responder verify those signatures before accepting the session, an attacker cannot substitute public values without also forging a valid RSA signature. The implementation uses a standard 2048-bit MODP group with generator 2 [2].

The use of fresh random exponents for each session is also an important strength of the design. Each conversation derives a new shared secret, and SHA-256 converts that secret together with the two nonces into fresh session-key material [4]. This is significantly stronger than using a long-term shared symmetric key for every message.

Even with these protections, several risks remain:

- The security of the authentication step depends on the authenticity of the RSA public keys themselves.
- The implementation does not maintain a global replay cache or persistent session-identifier database. Instead, replay protection is based on signed session nonces and per-session sequence counters.
- The implementation does not perform additional validation on received Diffie-Hellman public values before shared-secret computation.

For a production-ready system, trusted public-key distribution, stronger replay tracking, parameter validation, and hardened implementations would all be required.

Diffie-Hellman Attacks

Diffie-Hellman is particularly vulnerable to a man-in-the-middle attack, where an active attacker can sit between the sender and receiver as they intercept and replace the exchanged values g^a and g^b with their own values $g^{a'}$ and $g^{b'}$. This allows the attacker to compute both shared keys, as they know a' and b' and can see the exchanged values. Consequently, instead of the legitimate shared key g^{ab}, the attacker establishes two keys, allowing them to decrypt and re-encrypt messages in both directions.

Overall Hardening Measures

The current prototype is useful for demonstrating secure communication principles, but several improvements would be required for deployment beyond a classroom project:

- Use strong default RSA parameters and padded signature schemes
- Replace the current handwritten CBC-based AES message layer with a standard authenticated-encryption mode such as AES-GCM [8]
- Add a trusted public-key distribution mechanism, such as certificates
- Validate peer Diffie-Hellman public values before shared-secret computation
- Track session identifiers or previously seen packets to strengthen replay protection
- Use constant-time cryptographic implementations to reduce side-channel risk

Conclusion

In conclusion, this project demonstrates how multiple cryptographic building blocks can be combined to create a secure messaging prototype. Diffie-Hellman provides fresh session establishment, RSA signatures authenticate the participants and protect packet integrity, SHA-256 derives session-key material, and AES-128 encrypts the message content. The resulting design addresses the main security goals of confidentiality, authentication, integrity, and freshness for communication over an untrused channel.

At the same time, the analysis shows that secure design depends not only on the choice of cryptographic algorithms, but also on parameter sizes, key distribution, replay handling, and mode of operation. The prototype succeeds as an educational implementation because it makes these ideas visible in code while also revealing further improvements are still needed to meet production-level security expectations.

Contributions

Member	Contributions
Alex Khoury	• Implemented RSA, RSA signatures, Diffie-Hellman, integration of codebase, number theory concepts, the secure communication, nonces, so on. • Wrote explanation of all that has been implemented. • Refinement of the report
Christopher Murdoch Egener	• Designed and implemented a FastAPI + WebSocket server for real-time secure messaging, including server.py (message relay), client.py (async session management), and chat_demo.py (interactive terminal demo). • Assisted with Python backend development and integration during the project. • Refactor & polish other contribution • Assisted in polishing and reviewing the report.
Hudson Lu	• Implemented AES-128 (Symmetric Encryption Algorithm) • Wrote in the implementation and security analysis report for AES. • Wrote in the security report about possible RSA & DH attacks. • Wrote the conclusion for the report. • Assisted in polishing and reviewing the report.
Jeremy Anderson	• Designed and prototyped a FastAPI-based secure messaging backend on the `features/fastapi` branch, including API endpoints, demo scripts and key setup utilities. • Contributed to the Introduction and System Design sections of the report. • Assisted with Python programming and backend integration during development. • Assisted in polishing and reviewing the report.
Max Pearce Basman	• Established the repository, dev environment tooling, and project report outline • Directed team meetings • Created CLI interface template • Implemented websocket communication and the application layer that uses it to send messages • Refactored and merged code contributions • Formatted report, diagram, and citations
Tianmu Yang	• Implemented a complete message sending-receiving process demonstration module. • Designed attack models and implemented an attack demonstration module. • Implemented CBC on top of AES to enhance system security. • Cleaned up the codebase. Improved the reporting language and standardized its format.

References

[1] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Transactions on Computer Systems*, vol. 2, no. 4, pp. 277–288, Nov. 1984, doi: 10.1145/357401.357402.

[2] T. Kivinen and M. Kojo, "More Modular Exponential (MODP) Diffie-Hellman groups for Internet Key Exchange (IKE)," Request for Comments RFC3526, May 2003. doi: 10.17487/RFC3526.

[3] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA Cryptography Specifications Version 2.2," Request for Comments RFC8017, Nov. 2016. doi: 10.17487/RFC8017.

[4] National Institute of Standards and Technology (US), "Secure hash standard," Washington, D.C., technical report NIST FIPS 180-4, 2015. doi: 10.6028/NIST.FIPS.180-4.

[5] National Institute of Standards and Technology (US), "Advanced Encryption Standard (AES)," Washington, D.C., technical report NIST FIPS 197-upd1, May 2023. doi: 10.6028/NIST.FIPS.197-upd1.

[6] N. Mouha and M. Dworkin, "Report on the block cipher modes of operation in the NIST SP 800-38 series," Gaithersburg, MD, technical report NIST IR 8459, Sept. 2024. doi: 10.6028/NIST.IR.8459.

[7] Neso Academy, "AES Security and Implementation Aspects." Accessed: Apr. 14, 2026. [Online]. Available: https://www.youtube.com/watch?v=oQuBvLAKUM8

[8] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC." Accessed: Apr. 12, 2026. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/38/d/final

[9] Computerphile, "AES Explained (Advanced Encryption Standard)." Accessed: Apr. 14, 2026. [Online]. Available: https://www.youtube.com/watch?v=O4xNJsjtN6E

[10] Neso Academy, "AES Encryption and Decryption." Accessed: Apr. 14, 2026. [Online]. Available: https://www.youtube.com/watch?v=4KiwoeDJFiA

[11] Neso Academy, "AES Round Transformation." Accessed: Apr. 14, 2026. [Online]. Available: https://www.youtube.com/watch?v=IpuvKyeCrvU